

The Rails PDF Stack Decision Guide

Four real options, what each is actually good at, and how to tell which one you want.

LIBRARY	ENGINE	PER DOC	IMAGE COST	ACCESSIBLE	MAINTAINED
wicked_pdf / pdfkit	wkhtmltopdf	slow	small	no	archived 2023
grover / ferrum	headless Chrome	slow	+ hundreds of MB	no	yes
prawn	its own Ruby DSL	fast	small	partial	yes
typstify	Typst (Rust)	about 4 ms	small	yes, PDF/UA-1	yes

Timings measured on an M-series Mac compiling the same styled A4 invoice 40 times. "Accessible" means whether the pipeline can emit a tagged PDF a screen reader can navigate.

Pick wicked_pdf if

You already have HTML templates that work and rewriting them is not worth it this quarter. Be clear-eyed: wkhtmltopdf was archived in 2023, so you are accepting an unmaintained WebKit fork with no security fixes. That is a real trade, and sometimes it is the right one.

Pick a Chrome-based gem if

Your document genuinely needs a browser. JS-rendered charts, a design system you cannot port, web fonts with complex shaping. Nothing else here runs JavaScript. You pay for it in image size, memory and cold starts, but you are buying something real.

Pick prawn if

You want precise coordinate control, or you are drawing rather than laying out a document. Prawn is mature, pure Ruby, and genuinely better than everything else here when you need to place things at exact points. The cost is that every layout is code.

Pick typstify if

You are starting fresh or willing to rewrite 2 to 5 PDF views. You want millisecond renders, no browser in your image, and tagged PDF/UA-1 output for accessibility requirements. The cost is the rewrite: there is no HTML to Typst converter and there will not be one.

If you are migrating, what it actually costs

1. Write the serializer first. Everything the view interpolates becomes a key, formatted in Ruby.
2. Port the page frame. Paper, margins, header and footer become one set page(...) call.
3. Port the type, and vendor the font. A family your image cannot supply is the classic production surprise.
4. Port the body. Tables and grids map almost one to one. A real invoice went from 82 lines of ERB to 55 of Typst.

Most applications have two to five PDF views. Each is a sitting, not a project.

What typstify does not do

- typstify is Rails only. Sinatra, Hanami and plain Rack are not supported.
- There is no HTML to Typst converter. You rewrite each PDF view by hand. Most apps have 2 to 5 of them.
- Ruby threads will not speed up rendering. The binding holds the GVL, so use processes for throughput.
- The 4 ms figure needs ignore_system_fonts turned on. The default is 58 ms because Typst scans OS font directories on every render.

Written by the author of typstify. These are the limitations worth knowing before you choose it.